

Calculadora Digital de 3 Bits

Walter Alfaro Ulate, José Pablo Herrera Vargas, Santos Alexander Lopez Valles

Resumen—Este trabajo describe el diseño, implementación y validación de una calculadora digital de 3 bits utilizando el lenguaje de descripción de hardware Verilog HDL. El sistema permite operaciones de suma, resta y multiplicación sin signo utilizando módulos de combinación aritmética y una unidad aritmética integrada (ALU). Además, se desarrolló un módulo de conversión BCD binario y un controlador multiplexado de siete segmentos para mostrar los resultados en la FPGA basada en la arquitectura Artix-7. El siguiente flujo de diseño incluyó modelado RTL, simulación funcional, síntesis y validación experimental. También se analizó el consumo energético del sistema, el cual alcanzó una potencia total de 0.141 W, utilizando en su mayor parte la potencia estática del propio equipo. Los resultados experimentales confirman el correcto funcionamiento del sistema y demuestran la efectividad del diseño propuesto.

Palabras clave—Verilog HDL, lógica digital, ALU, FPGA, arquitectura RTL, display de siete segmentos, conversión BCD, potencia dinámica, potencia estática.

I. INTRODUCCIÓN

Este proyecto desarrolla una calculadora digital capaz de realizar operaciones de suma, resta y multiplicación sin signo en operandos de 3 bits. La implementación se realizó utilizando Verilog HDL siguiendo un flujo de diseño estándar que consiste en modelado RTL, simulación funcional, síntesis y validación experimental en FPGA.

El uso de HDL tiene ventajas significativas al permitir que la estructura del hardware se exprese de manera modular, portátil y comprobable, como se describe en el trabajo de LaMeres [3]. Las operaciones aritméticas desarrolladas se basan en los principios clásicos de lógica combinacional y secuencial descritos por Mano [4], mientras que la eficiencia y organización del hardware utilizado para la multiplicación está relacionada con las pautas aritméticas orientadas a aplicaciones discutidas por [1].

El sistema final integra módulos aritméticos, lógica de control, conversión numérica y una interfaz de pantalla que utiliza una pantalla multiplexada de siete segmentos. También incluye módulos secuenciales para generar señales de reloj derivadas del reloj maestro de la FPGA. Este artículo describe el marco teórico, la arquitectura del sistema, los resultados de simulación y validación física y el análisis energético del proyecto.

II. MARCO TEÓRICO

Esta sección presenta los fundamentos conceptuales necesarios para el diseño del sistema: representación numérica, aritmética combinacional, modelado RTL, lógica secuencial, conversión numérica, y el uso de dispositivos FPGA como plataforma de implementación.

II-A. A. Representación y Aritmética Binaria

Los sistemas digitales representan información mediante valores binarios, lo cual permite robustez frente al ruido y simplicidad en la implementación del hardware [3]. Con operandos de 3 bits, pueden representarse valores entre 0 y 7. Las operaciones aritméticas utilizadas requieren anchos de palabra mayores: la suma puede generar valores entre 0 y 14, la resta puede resultar en valores negativos y la multiplicación sin signo produce valores entre 0 y 63.

Estas operaciones se basan en conceptos de propagación de acarreo, préstamos y productos parciales, ampliamente descritos en textos clásicos de lógica digital [4].

II-B. B. Aritmética Combinacional

El sumador paralelo tipo *ripple-carry* se forma mediante la concatenación de sumadores completos, cuya latencia depende de la propagación del acarreo entre etapas [4]. La resta puede implementarse mediante complemento a dos o mediante determinación de magnitud y signo, siendo esta última la empleada en el presente diseño.

La multiplicación se representa como una suma ponderada de productos parciales, operación que puede sintetizarse eficientemente para operandos pequeños utilizando lógica combinacional [1].

II-C. C. Modelado RTL y Flujo de Diseño

El diseño RTL (*Register Transfer Level*) describe el flujo de datos entre registros y las operaciones lógicas que los transforman. Las etapas de un flujo de diseño digital estándar incluyen: modelado en HDL, simulación funcional, síntesis, implementación y validación experimental [3]. Este enfoque permite que la descripción abstracta en Verilog se convierta en hardware real al ser sintetizada en una FPGA.

II-D. D. Lógica Secuencial y Generación de Reloj

El sistema incorpora lógica secuencial basada en flip-flops sincronizados al reloj global de 100 MHz de la tarjeta. Los divisores de frecuencia implementados mediante contadores permiten obtener relojes derivados de 1 Hz, 100 Hz y 1 kHz, utilizados para multiplexar el display y sincronizar las señales de usuario.

II-E. E. Conversión binario-BCD y Decodificación

Para visualizar los resultados en formato decimal, se convierte el valor binario de 6 bits en los dígitos de decenas y unidades mediante división entera entre 10. Luego, cada dígito BCD se decodifica en un patrón de siete segmentos mediante una ROM sintetizada. Estos decodificadores corresponden a dispositivos MSI ampliamente utilizados en sistemas digitales [4].

Una FPGA (*Field Programmable Gate Array*) es un dispositivo digital reconfigurable cuya arquitectura permite implementar circuitos lógicos personalizados mediante hardware programable. A diferencia de los microcontroladores, que ejecutan instrucciones secuenciales, las FPGAs implementan directamente circuitos paralelos, siendo ideales para sistemas aritméticos y arquitecturas altamente concurrentes [3].

En este proyecto se utiliza la tarjeta de desarrollo Digilent Nexys-4, la cual incorpora la FPGA **Xilinx Artix-7 XC7A100T-1CSG324C**. La Artix-7 pertenece a la familia de dispositivos de 28 nm de Xilinx y está compuesta por:

- **LUTs (Look-Up Tables)** de 6 entradas, usadas para implementar lógica combinacional como sumadores, multiplexores, comparadores y decodificadores.
- **Flip-Flops (FFs)**, utilizados para almacenar estados y construir lógica secuencial.
- **Slices**, unidades que agrupan LUTs y FFs.
- **Bloques DSP48E1**, utilizados para operaciones aritméticas de alta velocidad.
- **BRAM (Block RAM)**, utilizada para almacenamiento interno.
- **Red de interconexión programable**, responsable de conectar la lógica sintetizada.
- **Recursos de I/O**, que permiten interactuar con los switches, botones, LEDs y displays del Nexys-4.

El flujo de síntesis convierte la descripción RTL en una red lógica de LUTs, FFs e interconexiones. Posteriormente, la etapa de implementación se encarga de ubicar físicamente cada componente en los recursos del Artix-7 y de rutear las conexiones. Finalmente, la configuración resultante se carga como un bitstream al FPGA, permitiendo ejecutar el diseño directamente en hardware.

Esta capacidad de implementar circuitos aritméticos personalizados de forma paralela y determinista convierte a la FPGA Artix-7 en una plataforma ideal para validar el funcionamiento del sistema de cálculo construido en este proyecto.

III. DISEÑO DEL EXPERIMENTO

III-A. A. Arquitectura General

El sistema integra módulos aritméticos (suma, resta y multiplicación), una ALU, un convertidor binario-BCD, divisores de frecuencia y un controlador de display.

III-B. B. Diagrama de Bloques del Sistema

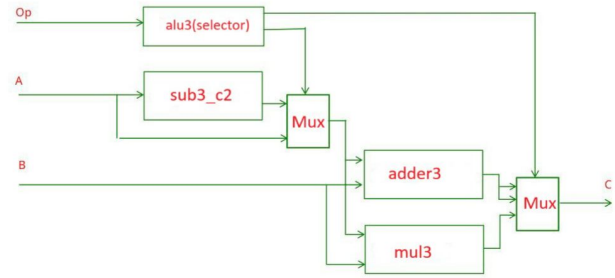


Figura 1: Diagrama de bloques del sistema aritmético implementado.

III-C. C. Módulos Aritméticos

El sumador `adder3` implementa un sumador paralelo ripple-carry. La resta se realizó mediante cálculo de magnitud y signo. La multiplicación usa una red combinacional sintetizada por Vivado.

III-D. D. Conversión y Visualización

El módulo `bin_to_bcd_0_63` obtiene decenas y unidades. El módulo `sevseg_two_digits` multiplexa el display.

III-E. E. Divisores de Frecuencia

Se generaron señales derivadas de 1 Hz, 100 Hz y 1 kHz mediante contadores.

III-F. F. Descripción de los Módulos del Sistema

El sistema completo fue desarrollado siguiendo una arquitectura modular en Verilog HDL. Cada módulo cumple una función específica dentro del flujo de datos entre los operandos y la salida final de la ALU. A continuación se describen los módulos principales.

III-F1. 1) `top_alu_nexys4.v` — Módulo Principal (Top-Level): Este módulo constituye el núcleo de integración del sistema y es el único que se sintetiza directamente sobre la FPGA. Sus funciones principales incluyen:

- Conectar switches, botones, LEDs y el display de siete segmentos de la tarjeta Nexys-4.
- Instanciar la ALU (`alu3`), el divisor de reloj (`clk_divider`) y el controlador de displays.
- Mostrar en tiempo real las entradas A, B y la operación seleccionada.
- Visualizar el resultado de la ALU y el bit de signo en caso de resta.
- Multiplexar los ocho dígitos del display.

III-F2. 2) sub3_c2.v — Módulo de Resta (Complemento a 2): Implementa la operación de resta entre operandos de 3 bits. Su funcionamiento incluye:

- Generación del complemento a 2 del operando B.
- Selección de la dirección de resta (A-B o B-A) mediante la señal `sub_dir`.
- Uso interno del sumador `adder3` para realizar la operación.

Produce una salida de 4 bits, incluyendo el bit de préstamo (*borrow*).

III-F3. 3) adder3.v — Sumador Completo de 3 Bits: Es un sumador paralelo de tres bits, formado por la concatenación de tres módulos `full_adder`. Realiza la operación:

$$A + B \rightarrow \text{salida de 4 bits (incluye acarreo)}$$

III-F4. 4) full_adder.v — Sumador de 1 Bit: Cada sumador elemental recibe:

- bit A,
- bit B,
- acarreo de entrada.

Produce:

- bit de suma,
- acarreo de salida.

III-F5. 5) mul3.v — Multiplicador de 3 Bits: Implementa la operación:

$$A \times B$$

donde ambos operandos son de 3 bits, generando un resultado de 6 bits. Este módulo es utilizado por la ALU cuando `op = 2'b10`.

III-F6. 6) alu3.v — Unidad Aritmética-Lógica: Es el módulo central del sistema. Sus funciones incluyen:

- Llamar al módulo de suma (`adder3`).
- Llamar al módulo de resta (`sub3_c2`).
- Llamar al multiplicador (`mul3`).
- Seleccionar la salida correcta según la señal `op`.

Genera la salida de 6 bits (`Y6`) y el bit de signo en caso de resta.

III-F7. 7) clk_divider.v — Divisor de Frecuencia: Genera señales derivadas del reloj de 100 MHz provisto por la tarjeta Nexys-4. Produce señales a frecuencias más bajas (1 Hz, 100 Hz, 1 kHz) necesarias para:

- Multiplexado del display,
- Contadores internos,
- Señales de temporización estable.

III-F8. 8) sevseg_encode.v — Codificador para 7 Segmentos: Convierte un dígito BCD (0-9) en un patrón de activación para un display de siete segmentos de cátodo común, utilizando niveles activos en bajo.

III-F9. 9) sevseg_two_digits.v — Controlador de Dos Dígitos: Se encarga del multiplexado dinámico de dos dígitos del display:

- Controla ánodos.
- Controla patrones de segmentos.

III-F10. 10) bin_to_bcd_0_63.v — Conversor Binario a BCD: Convierte un número binario entre 0 y 63 en:

- Dígito de decenas (0-6),
- Dígito de unidades (0-9).

Es necesario para visualizar el resultado de la ALU en formato decimal.

IV. RESULTADOS

IV-A. A. Validación Funcional

Cuadro I: Validación de operaciones aritméticas

A	B	Op	Esperado	Obtenido
3	2	Suma	5	5
5	1	Resta	4	4
1	6	Resta	-5	-5
7	7	Mult	49	49

IV-B. B. Latencia Funcional y Crítica

- Frecuencia del reloj: 100 MHz
- Período: 10 ns
- Latencia funcional: **10 ns**
- Latencia crítica (Vivado): **64.98 ns**

Cuadro II: Resumen de Potencia del Sistema

Parámetro	Valor (W)
Potencia dinámica	0.044
Potencia estática	0.097
Potencia total	0.141

IV-C. C. Potencia

Cuadro III: Resumen de potencia del sistema

Parámetro	Valor (W)
Potencia dinámica	0.044
Potencia estática	0.097
Potencia total	0.141

IV-D. D. Parámetros Térmicos

Cuadro IV: Parámetros térmicos (Vivado)

Parámetro	Valor
Temperatura de unión	25.6 °C
Temperatura ambiente máxima	84.4 °C
Resistencia térmica TJA	4.6 C/W

V. IMPLEMENTACIÓN EN FPGA

V-A. A. Herramienta de Síntesis

El diseño fue implementado utilizando **Xilinx Vivado 2023.1**. También utilizamos de **Chat GPT** para la explicación de ciertas partes del programa y para la ayuda de la estructura del proyecto.

V-B. B. Recursos Utilizados

Cuadro V: Recursos utilizados en la FPGA Artix-7 XC7A100T

Recurso	Uso	Total
Slice LUTs	87	63 400
Slice Registers	20	126 800
Slices	32	15 850
IO utilizados	43	210
BUFGCTRL	1	32

V-C. C. Repositorio del Proyecto

El código fuente completo del sistema está disponible en el siguiente repositorio público: <https://github.com/walle-au/Calculadora-Vivado>

VI. CONCLUSIONES

El proyecto desarrollado permitió diseñar, simular e implementar una calculadora digital de 3 bits utilizando Verilog HDL y la FPGA Artix-7 XC7A100T integrada en la tarjeta Nexys-4. La arquitectura modular empleada facilitó la integración de los módulos aritméticos (suma, resta y multiplicación), la conversión binario-BCD y el sistema de visualización multiplexado de siete segmentos.

Los resultados experimentales validan el correcto funcionamiento del sistema en hardware real. La latencia funcional de un ciclo de reloj (10 ns) y la latencia crítica reportada por Vivado (64.98 ns) demuestran que el diseño opera de manera estable bajo una frecuencia de 100 MHz, sin violaciones temporales. De igual forma, el análisis energético muestra un consumo total de 0.141 W, con predominancia de la potencia estática, lo cual es característico de los dispositivos FPGA modernos.

El uso de recursos de la FPGA fue mínimo: menos del 1 % de LUTs y flip-flops disponibles, confirmando que el diseño es altamente eficiente y deja un amplio margen para futuras ampliaciones. Entre estas posibles mejoras se encuentran la extensión a operandos de mayor tamaño, la incorporación de nuevas operaciones aritméticas o la integración dentro de un datapath más complejo, como una CPU elemental o un procesador embebido.

En conjunto, el proyecto permitió reforzar conceptos fundamentales de lógica combinacional, secuencial y modelado RTL, además de proporcionar experiencia directa en la síntesis e implementación de hardware digital en FPGA.

REFERENCIAS

- [1] F. de Dinechin y M. Kumm, Application-specific arithmetic: Computing just right for the reconfigurable computer and the dark silicon era, 2024a ed. Cham, Suiza: Springer International Publishing, 2024.
- [2] W. Liu y F. Lombardi, Eds., Approximate computing, 2022a ed. Cham, Suiza: Springer Nature, 2022.
- [3] B. J. LaMeres, Introduction to logic circuits & logic design with verilog, 2a ed. Cham, Suiza: Springer Nature, 2023.
- [4] M. Morris Mano, Diseño digital - 3b: Edición. Pearson Educación, 2005.